



RV College of
Engineering®

Unit I

Introduction to Processing Units

Go, change the world®

Unit 1: Syllabus

Introduction to Processing units

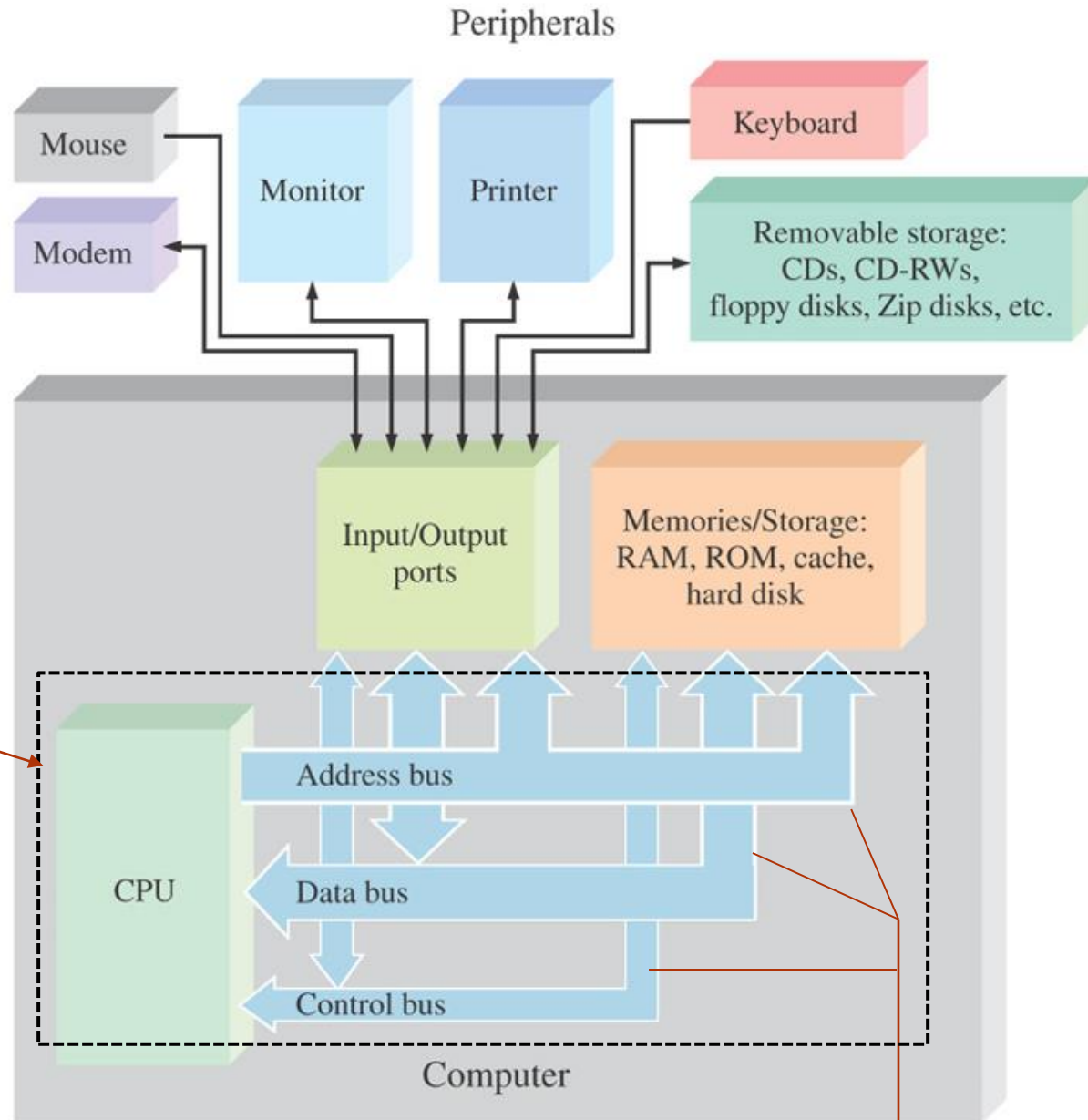
Computer System, Processor, Block diagram, Processor logic unit, Control unit, Instruction format, Assembly language, High level language, Embedded computing applications, Microcontroller, Instruction set architectures (CISC, RISC), Harvard and Von Neumann, Floating and fixed point

Introduction of controller families: 8-bit, 16-bit, 32-bit, 64-bit, ARM Processor families, Cortex A, Cortex R and Cortex M, Thumb 2 instruction set

Block diagram of a computer system

MP
IC

- Bus: Collection of wires



Bus

Source: T.L Floyd, "Digital Fundamentals", 9e

Block diagram of a computer

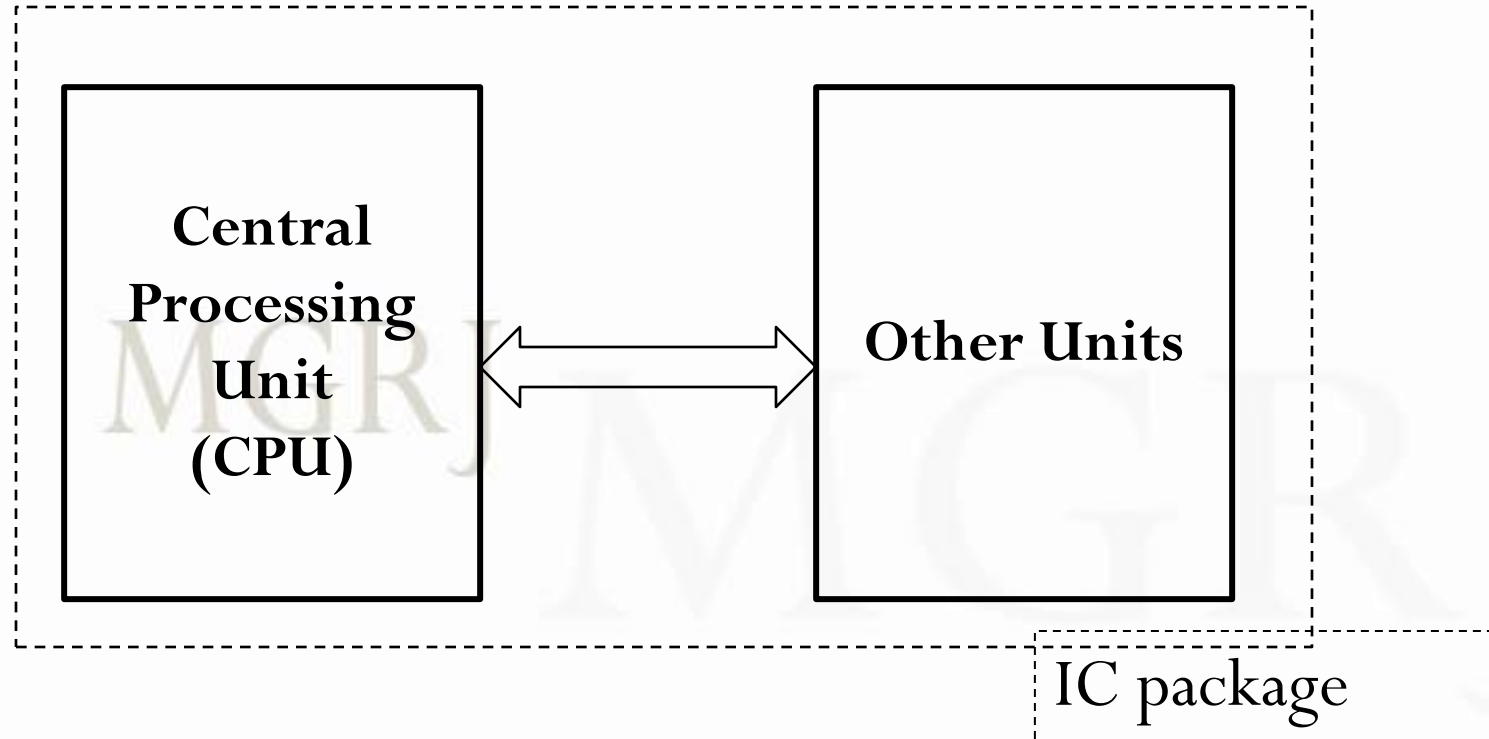
- All computer systems consists of basic functional blocks that include a CPU, memory and input/output ports.
- These blocks are connected together with three internal buses: Address bus, Control bus, Data bus collectively called as system bus.
- A port is a physical interface on a computer through which data is passed to and from peripherals.
- The memory includes program memory(ROM) to store instructions to be executed to solve a specific problem and data memory to store data during executing instructions.

MGRJ

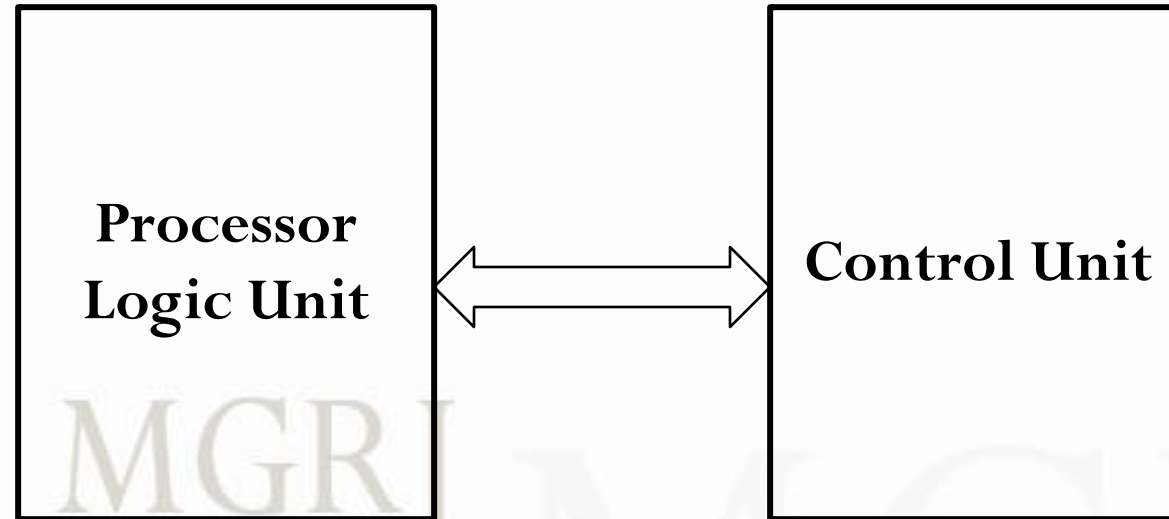
Processor

- A processor unit is that part of a computer system or digital system that implements the operations in the system.
- A Processor IC interfaced with many other components to realize computer system.
- The user is expected to give instructions to the processor by writing a program in assembly or high level language.

Processor: Block Diagram



CPU: Block diagram



- Processor logic unit is a digital circuit capable of performing *different operations*, which are controlled by *control unit* by generating *control signals*.

Processor Logic Unit(PLU)

- In most of the processors, the different operations are implemented by means of arithmetic & logical operations.
- The processor logic unit consists of circuits to implement simple basic operations like add, shift, etc...
- The other operations are generated by using basic operations available with the support of control unit.

E.g. Multiplication operation is generated by repeated add instruction/ Shift instruction

- Processor logic unit consists of circuits to implement Arithmetic & Logic operations. These circuits are called as Arithmetic & Logic Unit(ALU).

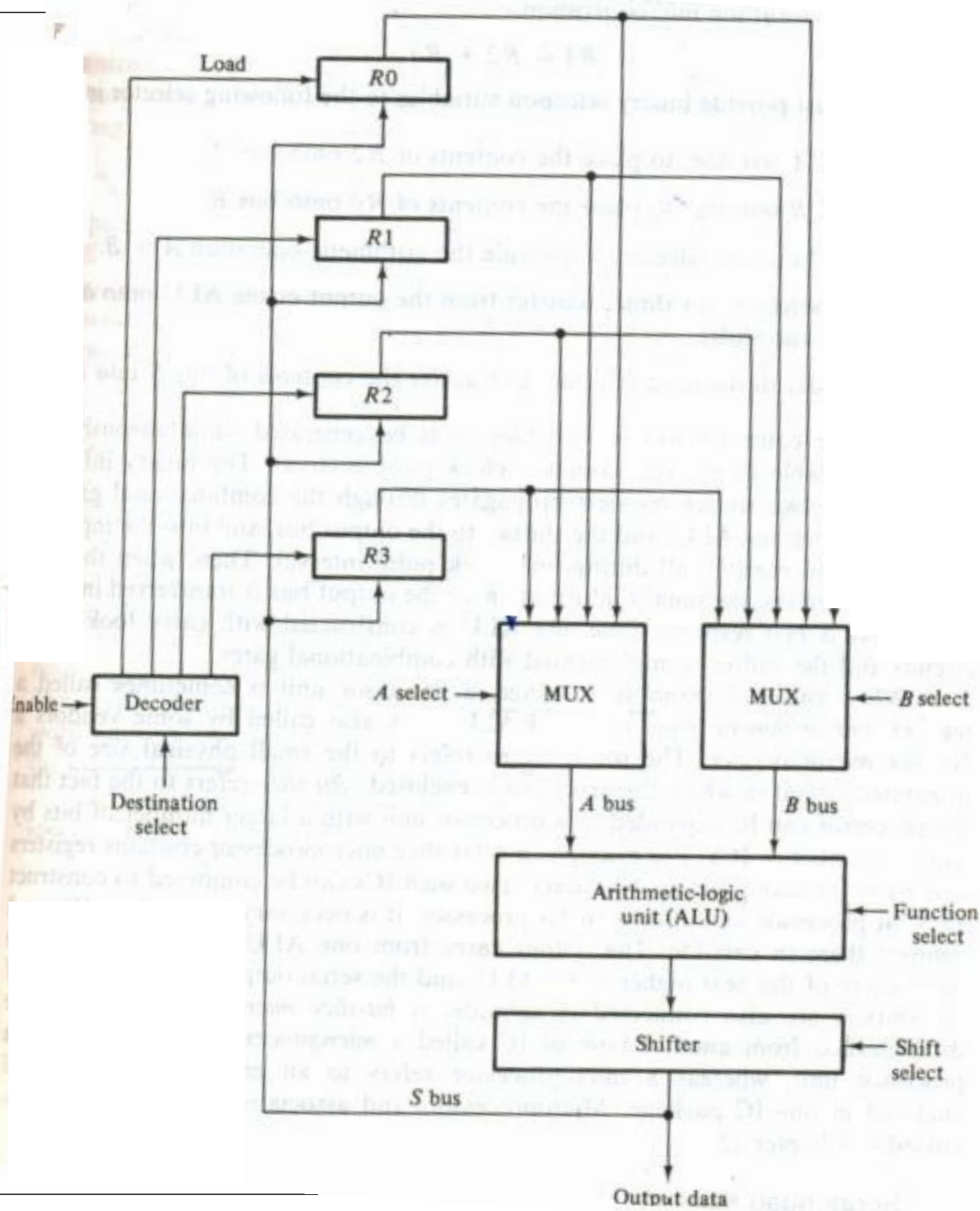
MGRJ

- The ALU receives the information(operands) from the registers and performs a given operation as specified by control unit.
- The operation to be performed (instruction) by CPU is specified by the user by writing programs.

E.g. C program.

- So, the PLU consists of ALU(additional digital circuits also) & registers connected by buses.

Simple PLU: Bus Organization



Bus Organization

- Different units are connected by buses: Bus organization
- Bus organization is also called as **data path architecture** because different functional units & their connectivity through buses is specified.
- Each register is connected to two multiplexers(MUX) to form input buses A & B.
- The input buses A & B are applied to a ALU.
- The function selected in the ALU determines the particular operation that is to be performed.
- The result of the operation goes through the output bus S into the inputs of all registers.
- The shift operation is implemented in the shifter.

Bus Organization: An Example

- Operands:
 - **MUX A selector**: One among 4 registers is to be connected to input bus A & B , hence at least two select line(bits) are required.

MUX –A

Select lines A1 A0	Register Selected
0 0	R0
0 1	R1
1 0	R2
1 1	R3

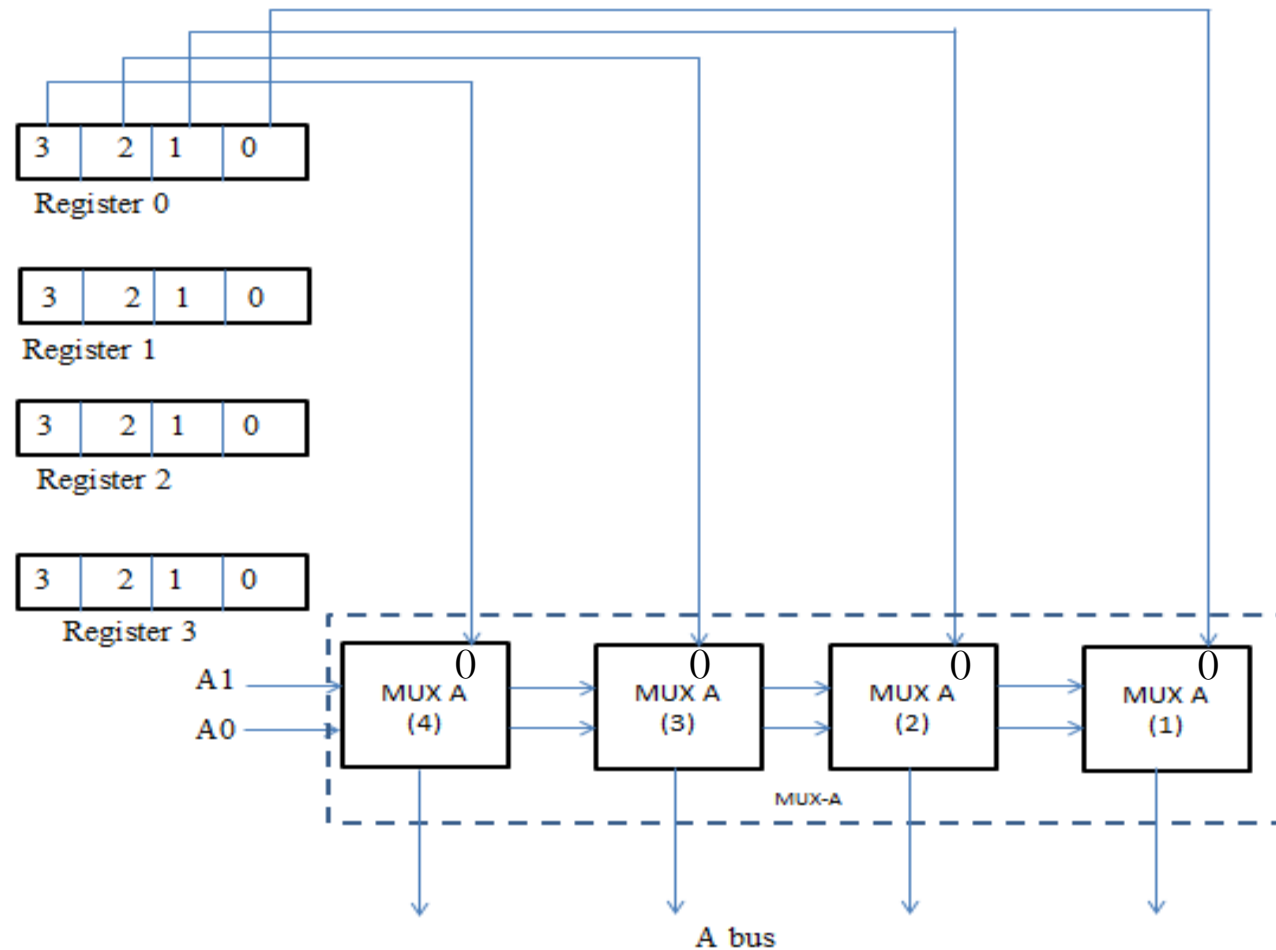
MUX-B

Select lines B1 B0	Register Selected
00	R0
01	R1
10	R2
11	R3

Number of multiplexers

- One MUX is required to connect a bit of register to ALU, the number MUX required is equal number of bits(size of ALU).

E.g: Size of ALU=4 bits $\Rightarrow$ Register Size=4 bits



Bus Organization: An Example

ALU

- Assume ALU performs 8 different operations.
- Hence, requires at least 3 bits of input to differentiate the operation.
- The ALU performs different operations according to table shown below.

Function Select			Operation
S2	S1	S0	
0	0	0	ADD
0	0	1	SUB
0	1	0	Complement
0	1	1	Increment
1	0	0	AND
1	0	1	OR
1	1	0	EX-OR
1	1	1	Decrement

Bus Organization: An Example

Shifter

- Assume if Shift select=1, shift the data,
shift select=0, no shift

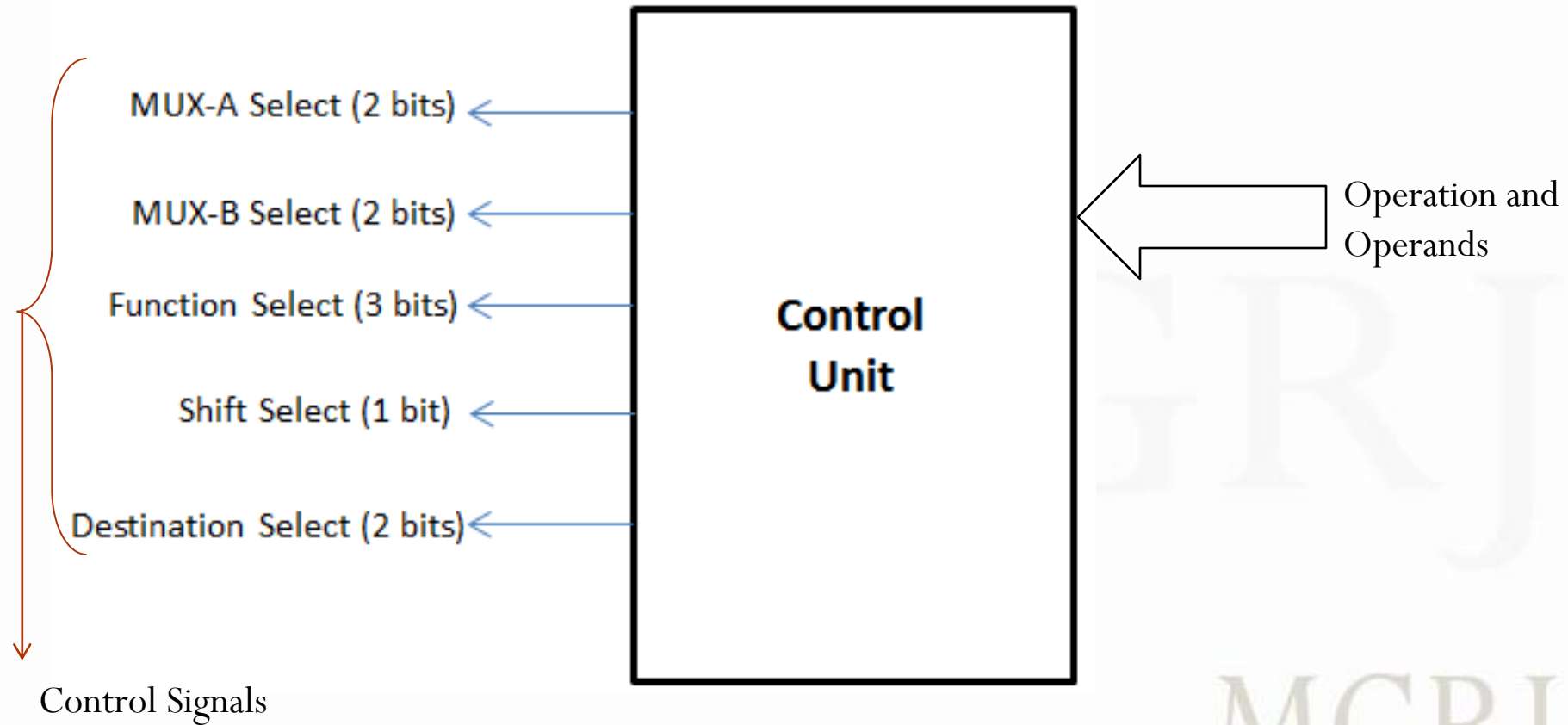
Decoder

- Decoder select a register to store data after operation.
- Assume decoder generate load signal to different register according to table shown below.

Destination select D1 D0	Register Selected
00	R0
01	R1
10	R2
11	R3

MGRJ

So the control unit is



Example:

- Assume: Operation to be performed is addition (by user)

$$R1 \leftarrow R1 + R2$$

- Operation is called as an *instruction*.
- R2 is source operand & R1 is source as well as destination

Different Control signals

- MUX A select: 01 to select R1
- MUX B select: 10 to select R2
- Function select: 000 for addition
- Shift select: 0 no shift
- Destination select: 01 to load data to R1

Example contd...

MGRJ

- The *macro operation(operation/Instruction)* addition involves many *micro operations*.
- Micro operations
 - Place contents of R1 onto bus A
 - Place contents of R2 onto bus B
 - Perform Addition in ALU
 - No shifting of data(transfer data)
 - Select Destination register

MGRJ

Example...

- So, for the operation assumed, the control signals to be generated are,

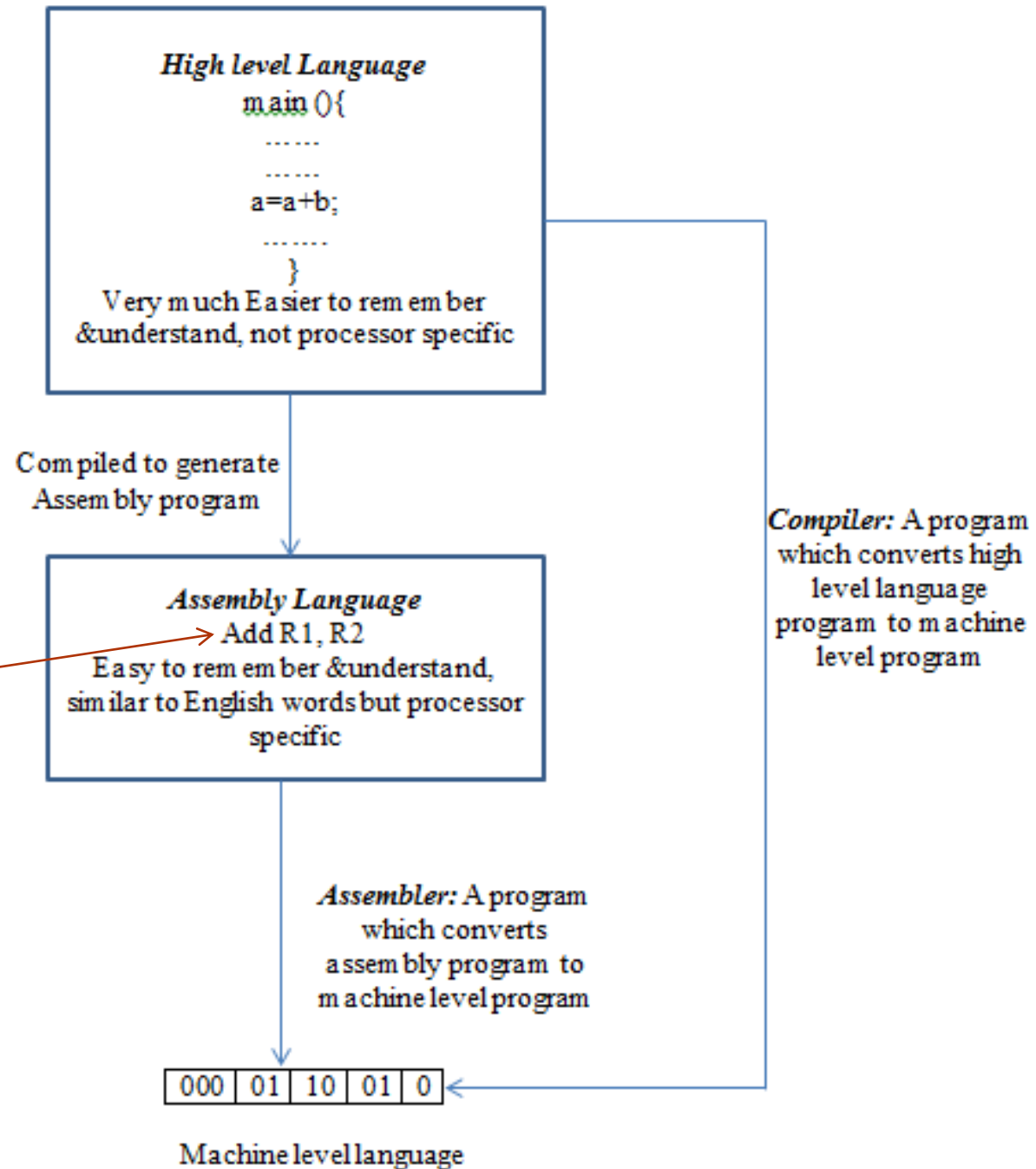
000	01	10	01	0
-----	----	----	----	---

Function Select	Source operand 1	Source operand 2	Destination	Shift Select
-----------------	------------------	------------------	-------------	--------------

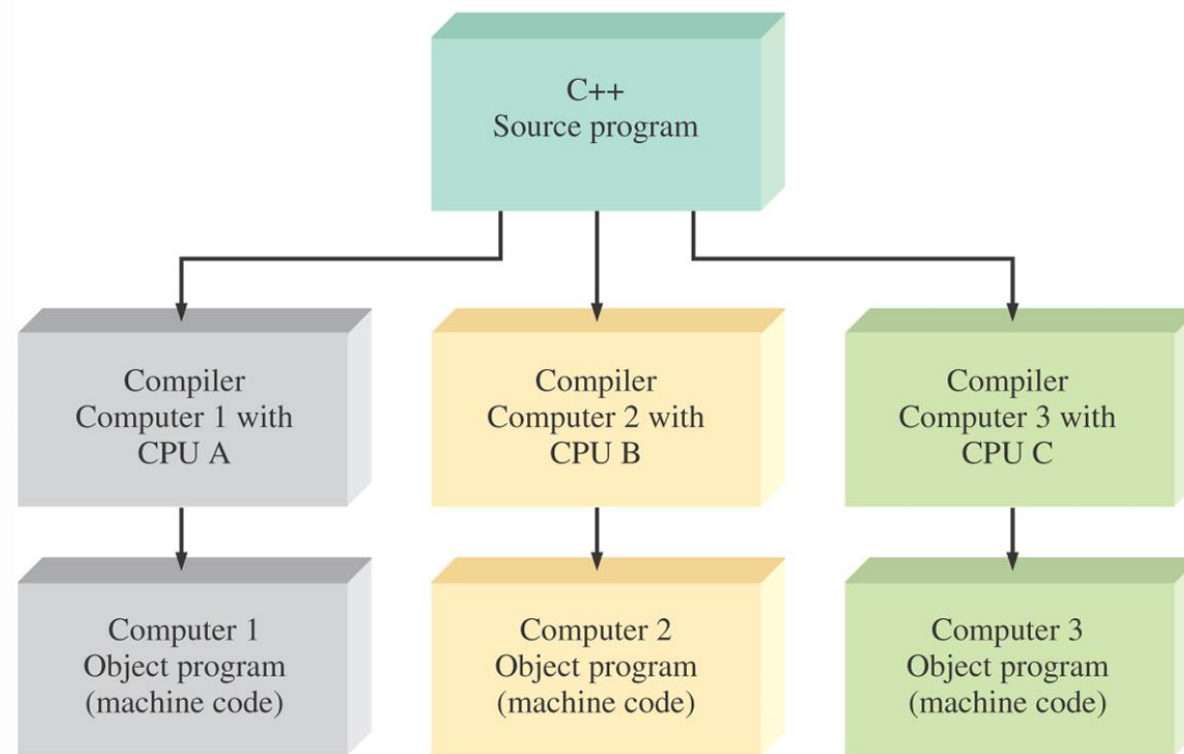
- The processor(Control unit & PLU) understands instructions in binary format called as *Machine(Processor) level language*.
- It is not possible for the user to give instructions in binary format.
- Hence, we have assembly language & high level language.

So,
how to write
instructions?

Mnemonics



MGRJ Machine Independent High level languages



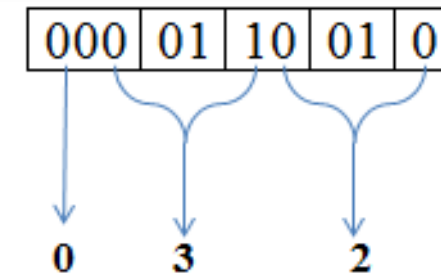
Source: T.L Floyd, "Digital Fundamentals", 9e

Example....

- Compiler & assembler generates **machine codes**, based on *instruction format* of the processor.
- **Instruction format** or **control word** of the PLU considered:

Function Select	Source operand 1	Source operand 2	Destination	Shift Select
-----------------	------------------	------------------	-------------	--------------

- Function select is also called as **Opcode(Operation Code)**
- Machine code of the operation considered:
32H (H-Hexadecimal)



- So, this is one byte(8 bits) instruction.
- Assembly instructions are called **instruction set** of processor.

Timing Sequence

- Each micro operation assumed takes some time for completion.
- The control signals are expected to generate in a sequence starting from source operand selection.
- Assume, different successive times instants during which control signals generated to complete the operation.
- T0 is time instant during which control signals for register selection are generated.

T0: Register selection (MUX A Sel=01, MUX B Sel= 10)

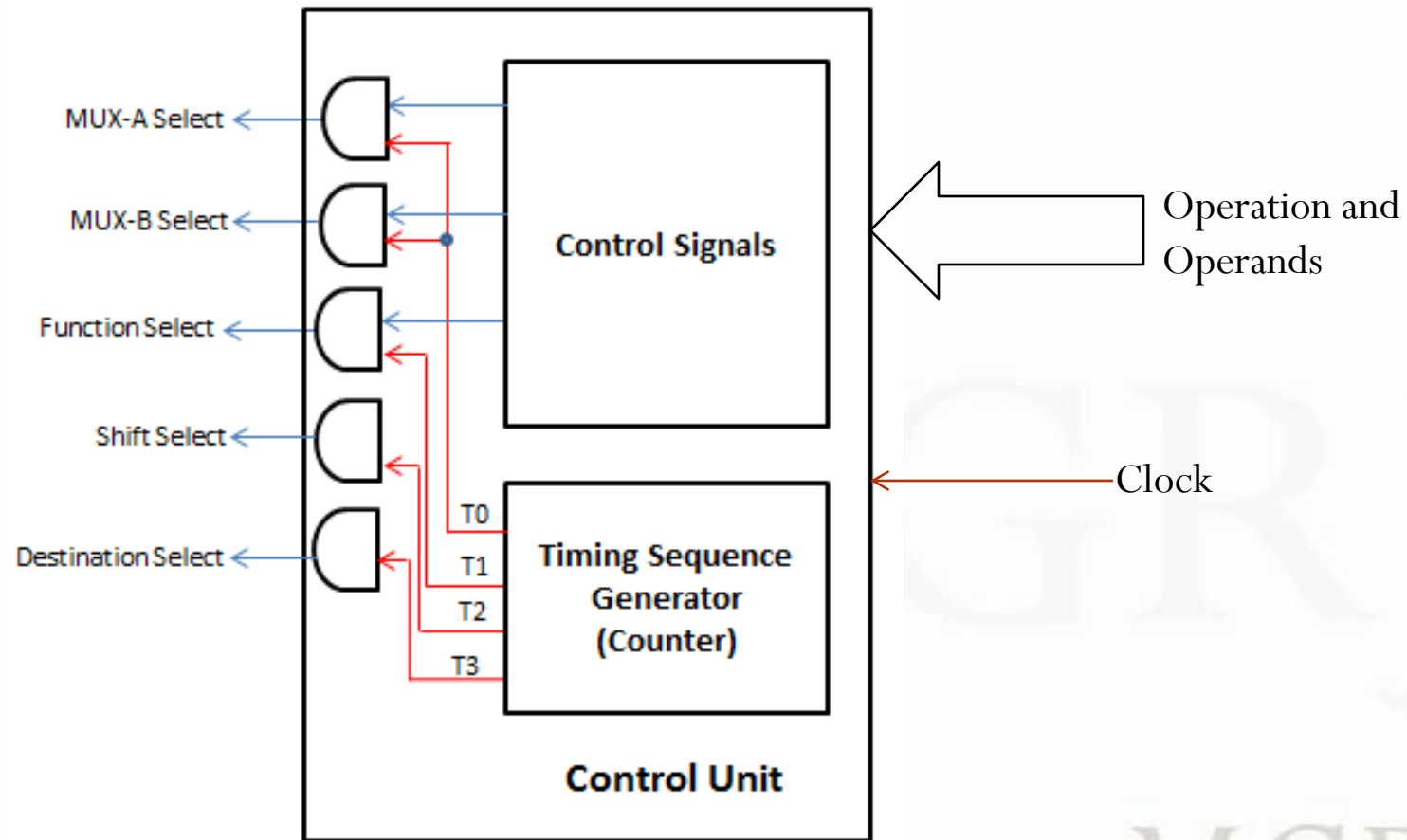
T1: Addition (Opcode=000)

T2: Shift(Shift Select=0)

T3: Destination Selection(Decoder input=01)

- To complete all micro operations, at least 4 time instants are required.
- The sequence of time instants form **timing sequence**.

Control unit modified



Questions?

- What is instruction code?
- What determines frequency of the clock?
- What is micro and macro instruction?
- What is control signal?
- What is mnemonics?
- What is an ALU size?
- What is a bus?

Control Unit(CU) Basics

- CU generates different control signals needed to perform different operations in data path.
- 2 ways to implement: **Hardwired & Micro programmed**
E.g: Consider a assembly program as follows for the PLU Considered.

(Note: Instructions & Mnemonics assumed are arbitrary.)



- Assembler directives are used to give information to assembler only, no machine codes are generated, hence called as pseudo instructions in the program.
(Assembler directives are similar to pre processor directives of high level languages).

Assembly Program with machine codes

Instruction	Machine Code (Binary)	Machine code (Hexadecimal)
SUB R2,R1	0011001100	0CCH
CPL R3	01011	0BH
AND R2,R3	1001011100	25CH
OR R2,R3	1011011100	2DCH

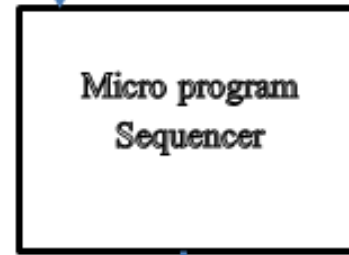
- If the sequence of machine codes to be generated are known, then a digital circuit can be designed to generate the machine codes => “***Hardwired***” ***Control Unit***
- So, hardwired control unit is predesigned hardware capable of generating one sequence of machine codes.

Control unit: Micro programmed

- The sequence of control signals(micro program) necessary to execute the different instructions are stored in ROM called *control ROM(Micro program memory / Program memory)*.
- To execute instructions, the control signals stored in the ROM can be accessed.
- The control signals read from the ROM are used to control the micro operations associated with different instructions to be executed at any time.
- The address of the next instruction is generated by a special hardware called **micro program sequencer**.

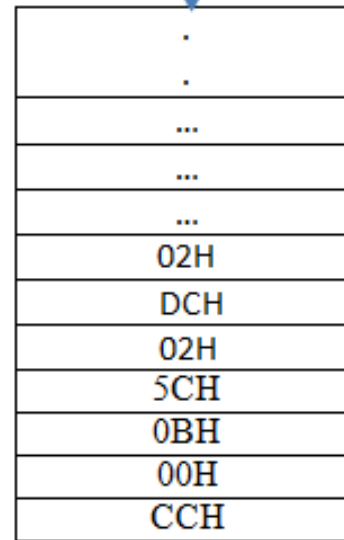
Micro programmed Control unit

Next address Information



Timing information

Status signals from
PLU



Program Memory

Control signals

Questions

1. Mention different functional units of a computer.
2. What is a processor?
3. What is Processor unit & Control unit ?
4. What is address bus, data bus, control bus?
5. What is address space of the processor?
6. What is the size of the memory supported by processor?

Questions

1. What is required in a processor to support memory?
2. What is data memory?
3. What is program memory?
4. What is chip select signal?
5. How many memory chips of size $4k \times 4$ are required to realize $8k \times 8$ memory?

Microprocessor (MP) & Microcontroller (MC)

MP

- A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions.
- It is a dependent unit. It requires the combination of other chips like Timers, Program and data memory chips, Interrupt controllers etc. for functioning.

MC

- A microcontroller is a highly integrated chip that contains a CPU, RAM, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports.
- It is a self contained unit and it doesn't require external Interrupt Controller, Timer, UART etc. for its functioning.

Microprocessor (MP) & Microcontroller (MC)

MP

- Doesn't contain a built in I/O port. The I/O Port functionality needs to be implemented with the help of external Programmable Peripheral Interface Chips.
- Targeted for high end market where performance is important.
- Most of the time general purpose in design and operation.
- Limited power saving options compared to microcontrollers.

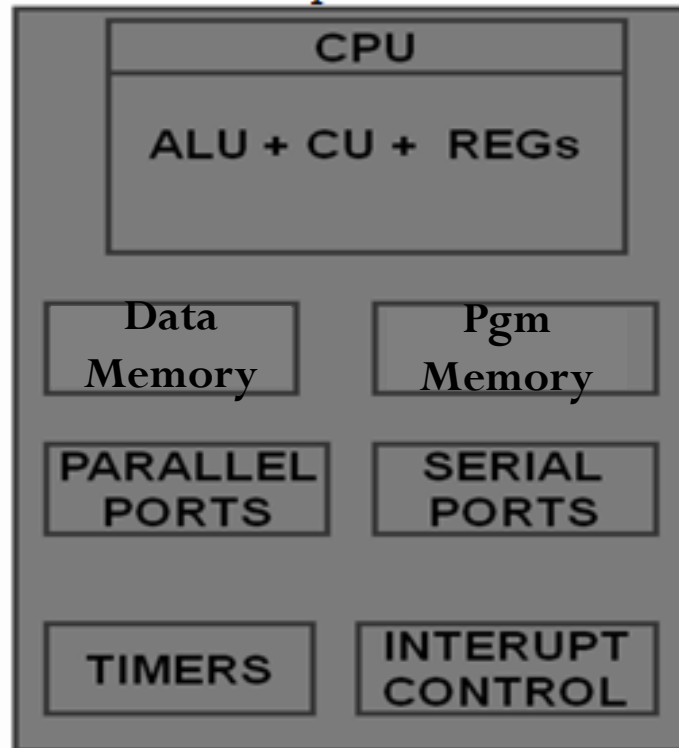
MC

- Most of the controllers contain multiple built-in I/O ports which can be operated as a single 8 or 16 or 32 bit Port or as individual port pins.
- Targeted for embedded market where performance is not so critical (At present this demarcation is invalid).
- Mostly application oriented or domain specific.
- Includes lot of power saving features.

Microprocessor (MP) & Microcontroller (MC)

MGRJ

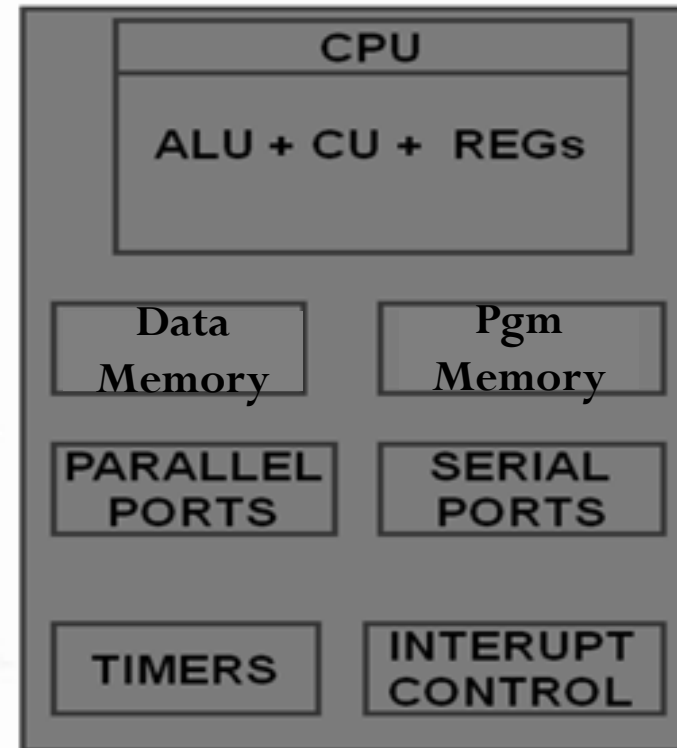
Microprocessor



On Single Board

- E.g: 8086, Intel I5, I7

Microcontroller



On Single Chip

- E.g: 8051, STM32F407VG

MGRJ

Instruction Set Architectures(ISAs)

- An **instruction set**, or **instruction set architecture (ISA)**, is the part of the processor architecture related to programming.
- All processors are supported by instruction set /instructions (Assembly instructions) which are dependent on organization of different components in PLU.
- Depending upon the way of supporting different instructions, the ISA is divided into
 - Reduced Instruction Set Computer(RISC)
 - Complex Instruction Set Computer(CISC)
- Other types of ISA
 - Very Long Instruction Word(VLIW), etc....

CISC & RISC Design Philosophy: CISC Vs RISC

CISC

- More number of instructions
- Instructions are complex to understand.
- Hardware support for many instructions (More silicon Usage)

A programmer can achieve the desired functionality with a single instruction which in turn provides the effect of using more simpler single instructions in RISC

- Clock cycles per instruction(CPI) is more.

RISC

- Lesser no. of instructions.
- Instructions are Easier to understand.
- Software support for many instructions/operations.
(Less silicon usage)

Programmer needs to write more code to execute a task since the instructions are simpler ones

- Clock cycles per instruction(CPI) is less.

CISC & RISC Design Philosophy: CISC Vs RISC

CISC

- Code density is more.
- Less number of registers.
- Memory to memory operations are supported.

Load & store operations in a instruction

- More number of addressing modes.
- Variable length instructions.
- Design of Pipelining is Complex.

RISC

- Code density is less.
- More number of registers.
- No memory to memory operations are supported.

Load & store operations not in a instruction (So called as load-store architecture)

- Less number of addressing modes.
- Fixed length instructions.
- Design of Pipelining is easier.

CISC & RISC Design Philosophy: CISC Vs RISC

CISC

- **Non Orthogonal Instruction Set**

All instructions are not allowed to operate on any register and use any addressing mode. It is instruction specific.

- Examples: 8086

RISC

- **Orthogonal Instruction Set**

Allows each instruction to operate on any register and use any addressing mode.

- Examples: ARM, MSP 430, PIC

POWERPC, ATmega328P

NOTE: The fact is, the designers are not worried about the architecture(CISC/RISC). So, the features from both the architectures are mixed up to increase the performance(Increase speed & reduce memory consumption).

Questions

- What is code density?
- What is an orthogonal instruction set?
- Why CPI is less in RISC architecture?
- Which is the preferable control unit to support complex operations ?
- What is the advantage/disadvantage of fixed length instructions?
- What is hardware support for an instructions?

Von Neumann & Harvard Architecture

- This classification is based on processor architecture design to support memory.

- **Address Space:**

- No. of locations a processor/controller can address.

E.g: **8086**: Address bus=20 bits, so address space is 1 Mb

(00000H-FFFFFFH)

8051: Address bus=16 bits, so address space is 64 Kb

(0000h-FFFFh)

ARM Cortex M4: Address bus= 32 bits, So address space is

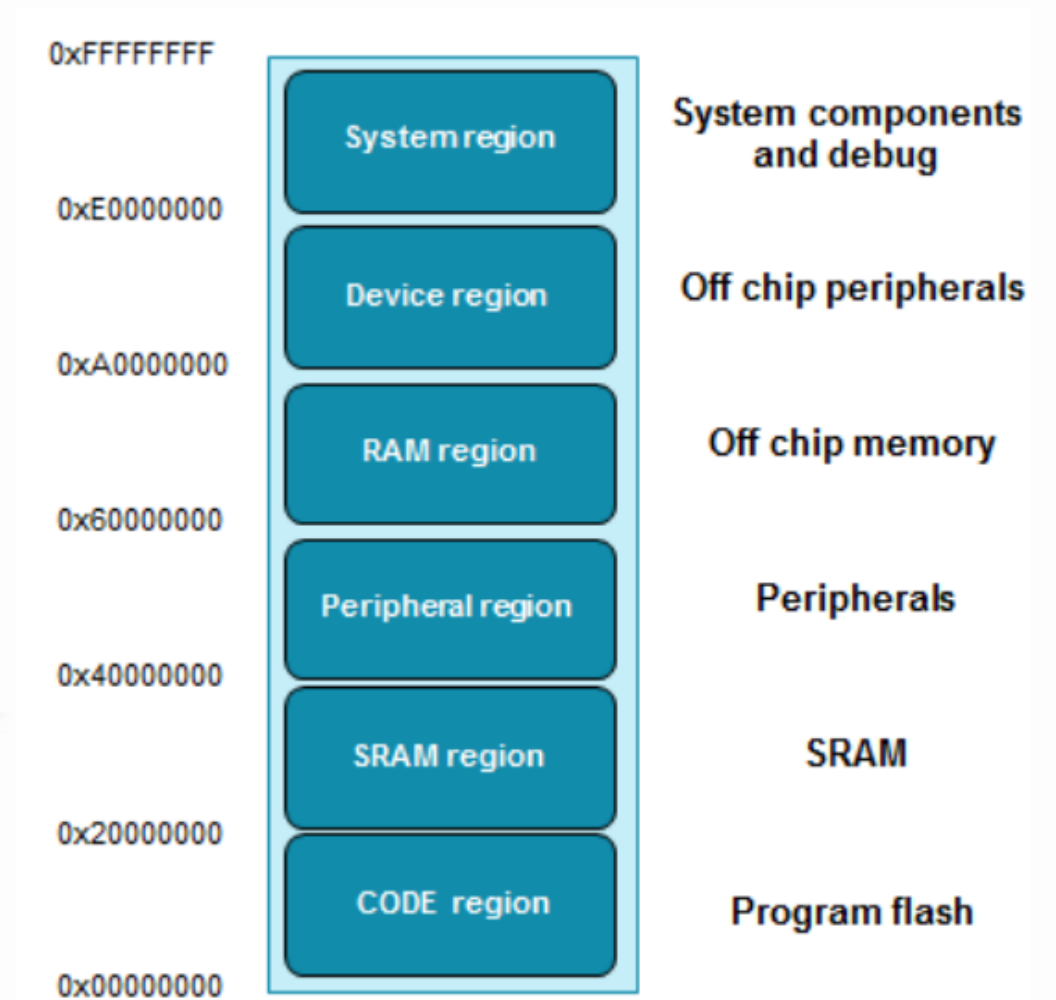
4GB(00000000h-FFFFFFFFh).

Von Neumann/Princeton Architecture

- In this architecture, *address space is shared* between program memory & data memory.

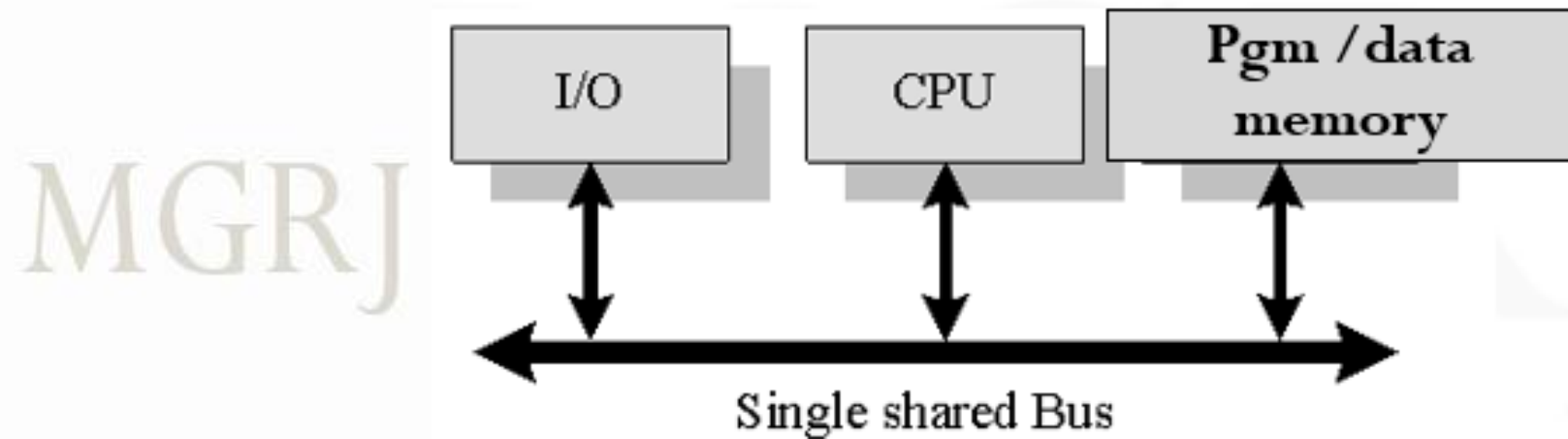
E.g: STM32F407VG(based on Cortex M4)

- Total Address space is 4GB(32 bit address)
- The address space is shared between code (Program flash), data(SRAM) IO(peripheral),etc.



Von Neumann/Princeton Architecture....

- Single shared bus(Address, data & control: System bus) for Instruction and Data fetching.



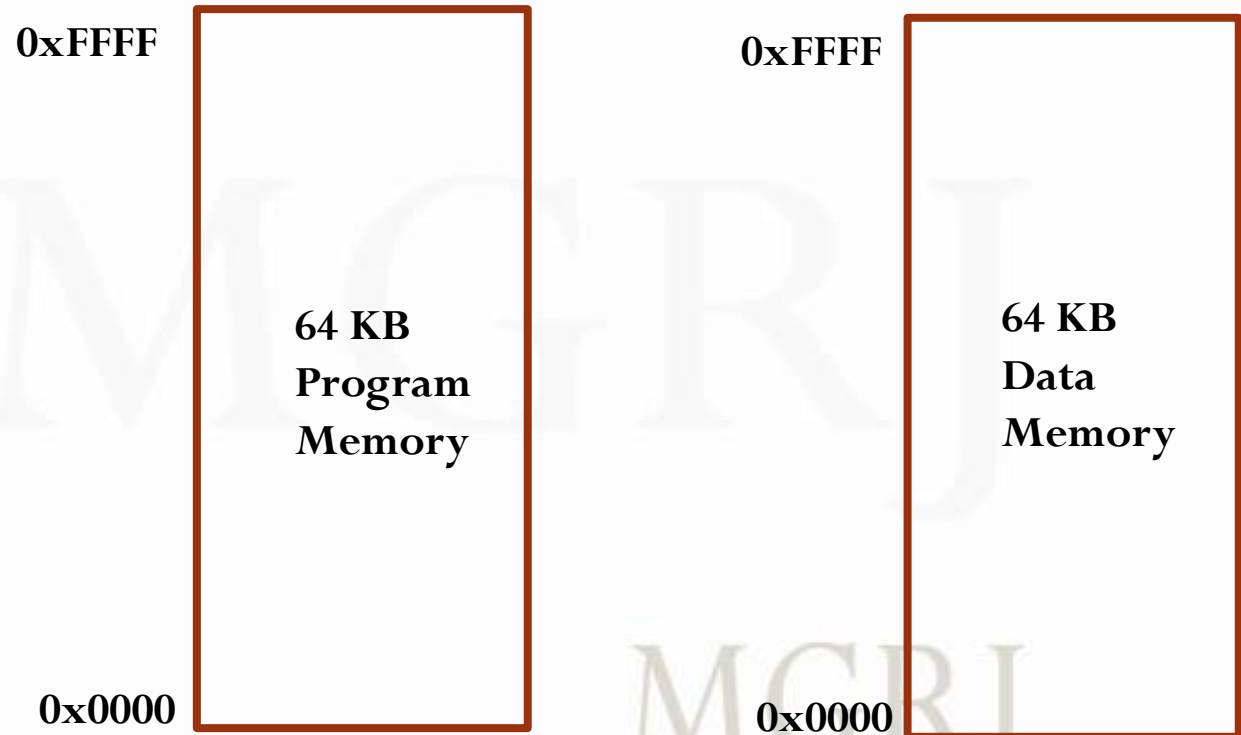
- The speed of execution is less because sharing of bus.
- The complexity of design of processor is less because single bus.

Harvard Architecture

- In this architecture, *address space is not shared* between program memory & data memory.

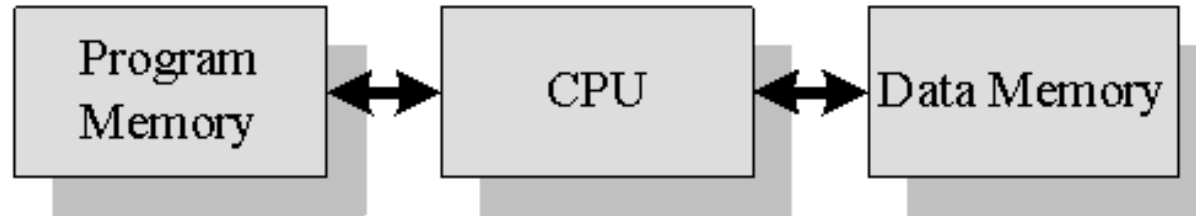
E.g:8051

- Total address space for program memory is 64KB & for data memory is 64KB.
- Program memory & data memory locations are *separate*.



MGRJ

- Separate buses for Instruction and Data fetching.



- The speed of execution is more because separate buses.
- The processor design is complex.

MGRJ

MGRJ

Final Note:

- Though STM32F407VG is Von Neumann address space, the architecture is supported with separate buses for program and data space to increase the speed.
- In fact, many controllers are based on separate bus architecture for program and data memory.
- Von Neumann address space and Harvard bus architecture

MGRJ

Endianness

- Endianness refers to the sequential order in which bytes are arranged into larger numerical values when stored in memory.

- Wiki

- **Little-endian Operation**

Base Address+ 0	Byte 0	Byte0	0x20000(Base Address\$
Base Address+ 1	Byte 1	Byte1	0x20001(Base Address+1)
Base Address+ 2	Byte 2	Byte2	0x20002(Base Address+2)
Base Address+ 3	Byte 3	Byte3	0x20003(Base Address+3)

- E.g:STM32F407VG

Endianness...

- **Big-endian Operation**

Base Address+ 0	Byte 3	Byte3	0x20000(Base Address\$
Base Address+ 1	Byte 2	Byte2	0x20001(Base Address+1)
Base Address+ 2	Byte 1	Byte1	0x20002(Base Address+2)
Base Address+ 3	Byte 0	Byte0	0x20003(Base Address+3)

E.g. Motorola 68000

Fixed point and Floating point Processors

- Processing can be separated into two categories: **fixed point and floating point**.
- These designations refer to the format used to store and manipulate numeric representations of data.
- Fixed-point CPUs are designed to represent and manipulate integers – positive and negative **whole numbers**.
- Floating-point CPUs represent and manipulate **rational numbers** in a manner similar to scientific notation IEEE 754 (Sign, mantissa and an exponent).

Fixed point and Floating point ...

- The term '**fixed point**' refers to a fixed number of digits after the decimal point.

E.g. 123.45, 1234.56, 12345.67

- With '**floating-point**' representation, the placement of the decimal point can 'float' relative to the significant digits of the number.

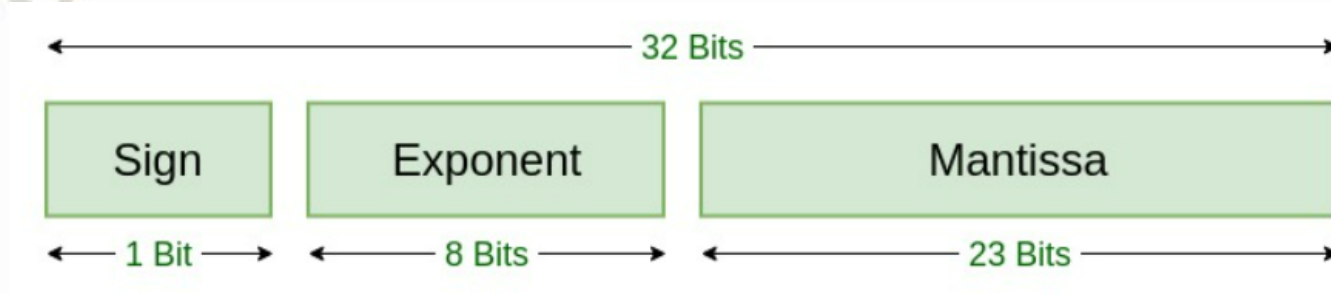
E.g. 1.234567, 123456.7, 0.00001234567, 12345670000000000

- The floating-point computation assures a much larger *dynamic range* - the largest and smallest numbers that can be represented.
- Floating-point processing yields much greater **precision**(accuracy) than fixed-point processing.

Fixed point and Floating point ...

- Certain processors are made fixed point where computations on floating point numbers are carried out by converting them to fixed representations.
- These fixed point processors are relatively economical and faster for some operations(integer operations).
- **STM32F205** is a fixed point ARM cortex M3 based microcontroller.
- The floating point processors are supported with dedicated floating point unit(FPU) to perform operations on floating point numbers by converting them to **single precision(32 bit)** and **double precision(64 bit)** IEEE 754 representation.
- **STM32F407VG** is a single precision floating point ARM cortex M4 based microcontroller.

- Single precision(SP) floating point number representation



- E.g: $85.125 = 1010101.001 = 1.010101001 \times 2^6$

Sign=0

Biased exponent = $127 + 6 = 133 = 10000101$

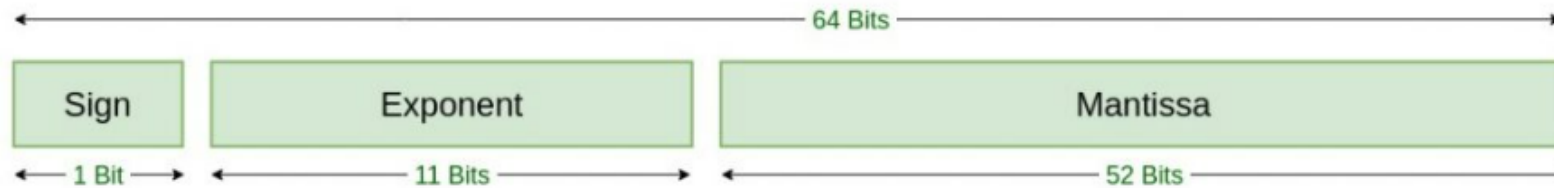
Normalized Mantissa = 010101001 (zeros are appended to make it 23 bits)

IEEE 754 format: SP

0 100 0010 1 010 1010 0100 0000 0000 0000 = 42AA4000H

Fixed point and Floating point

- Double precision(DP) floating point number representation



- $85.125 = 1010101.001 = 1.010101001 \times 2^6$

Sign=0

Biased exponent = $1023 + 6 = 1029 = 100\ 0000\ 0101$

Normalized Mantissa = 010101001 (zeros are appended to make it 52 bits)

IEEE 754 format: DP

0 100 0000 0101 0101 0100 1000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

= **0x4055480000000000H**

Introduction to controller families

- 8-bit MCUs are low pin count devices offer great advantage in power consumption compared to 16 bit and 32-bit counterparts.
- Typical Applications: Personal blood pressure monitors, pulse oximeters, heart rate monitors, RF/Zigbee or Wi-Fi interfaces, transceivers, smartcards, power windows of automobiles, smoke detectors, glass breakage detectors, thermostats, smart meters, etc.
- In addition, these devices found abundant usage in consumer products like TV remotes, geysers, smart rice cookers, washing machines, smart fans, etc.

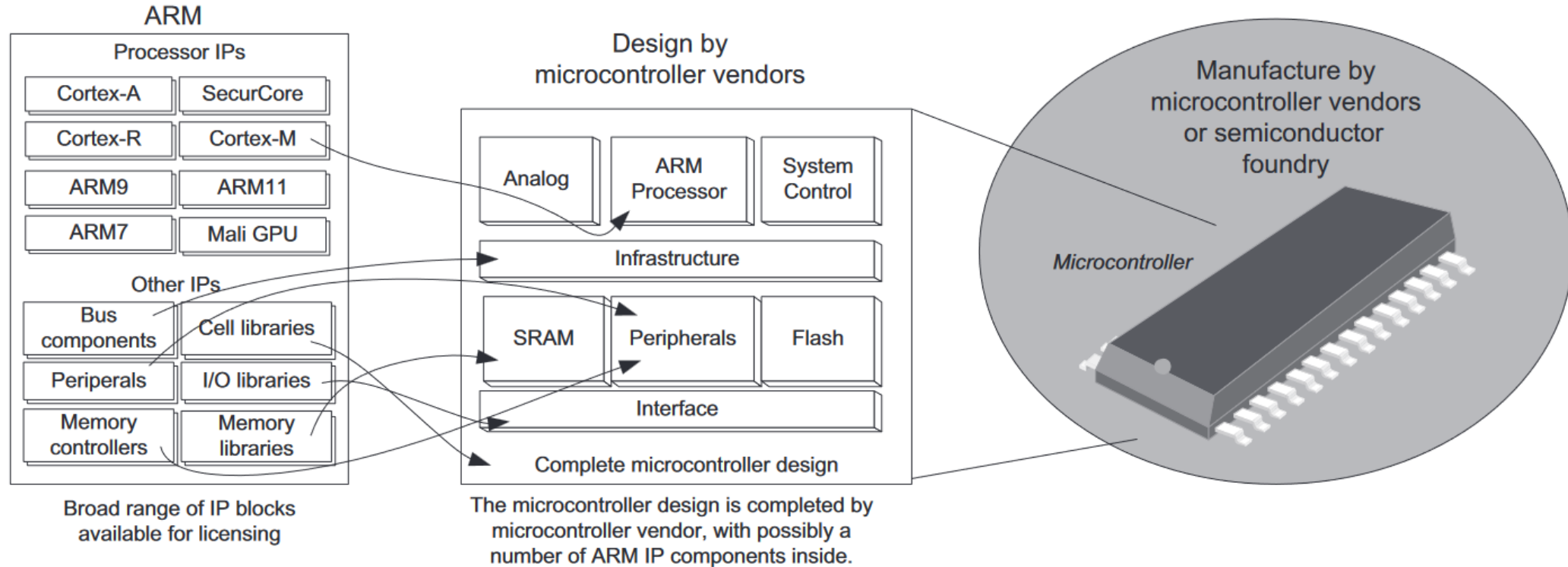
8 bit MCUs families:

- Intel 8051 family: 8051, 8031, 8052, 8032, AT89C51, AT89LV51 (Intel is not silicon provider).
- Microchip PIC® and AVR® Micro controllers: PIC16F, PIC18F series, AVR DD, AVR DA, AVR DB and ATtiny1627 families. (AVR is Atmel earlier)
- STMicroelectronics STM8 series: STM8S, STM8L, STM8AF and STM8AL
- Infineon 8FX Microcontroller: CY95F632H/K, CY95F633H/K, CY95F634H/K, CY95F636H/K (CY: Cypress is now Infineon)

What is Arm(earlier ARM)?

- The Arm holdings design 32/64 bit reduced instruction set computer (RISC) instruction set architecture (ISA) (?)
- ARM does not manufacture silicon.
- ARM's business is to sell IP cores, which licensees use to create microcontrollers and CPUs based on this core.
- IP cores:
 - Gate Netlist (Hard)
 - Synthesizable RTL code (Soft)

IPs Vs Silicon



- E.g: STM32F407VG(MCU used in lab by STMicroelectronics)

ARM Cortex M4 Core given ARM company

ARM Design Features

- Conditional execution of most instructions, reducing *branch overhead* and compensating for the lack of a *branch predictor(?)*.

- It avoids branch instructions when generating code for small if statements.

- In the C programming language, the loop is:

```
while (i != j)
```

```
{
```

```
    if (i > j)
```

```
        i -= j;
```

```
    else
```

```
        j -= i;
```

```
}
```

- In ARM assembly, the loop is:

```
loop :CMP Ri, Rj ; set condition "NE" if (i != j):
```

“GT” if (i > j) or “LT” if (i < j)

```
SUBGT Ri, Ri, Rj ; if "GT" (greater than), i = i-j;
```

```
SUBLT Rj, Rj, Ri ; if "LT" (less than), j = j-i;
```

```
BNE loop ; if "NE" (not equal), then loop
```

- The Conditional execution avoids the branches around the **then** and **else** clauses.

Design Features contd..

- Another feature of the instruction set is the ability to fold shifts and rotates into the data processing instructions. For example, the C statement:

$a += (j \ll 2);$

could be rendered as a single-word, single-cycle instruction on the ARM.

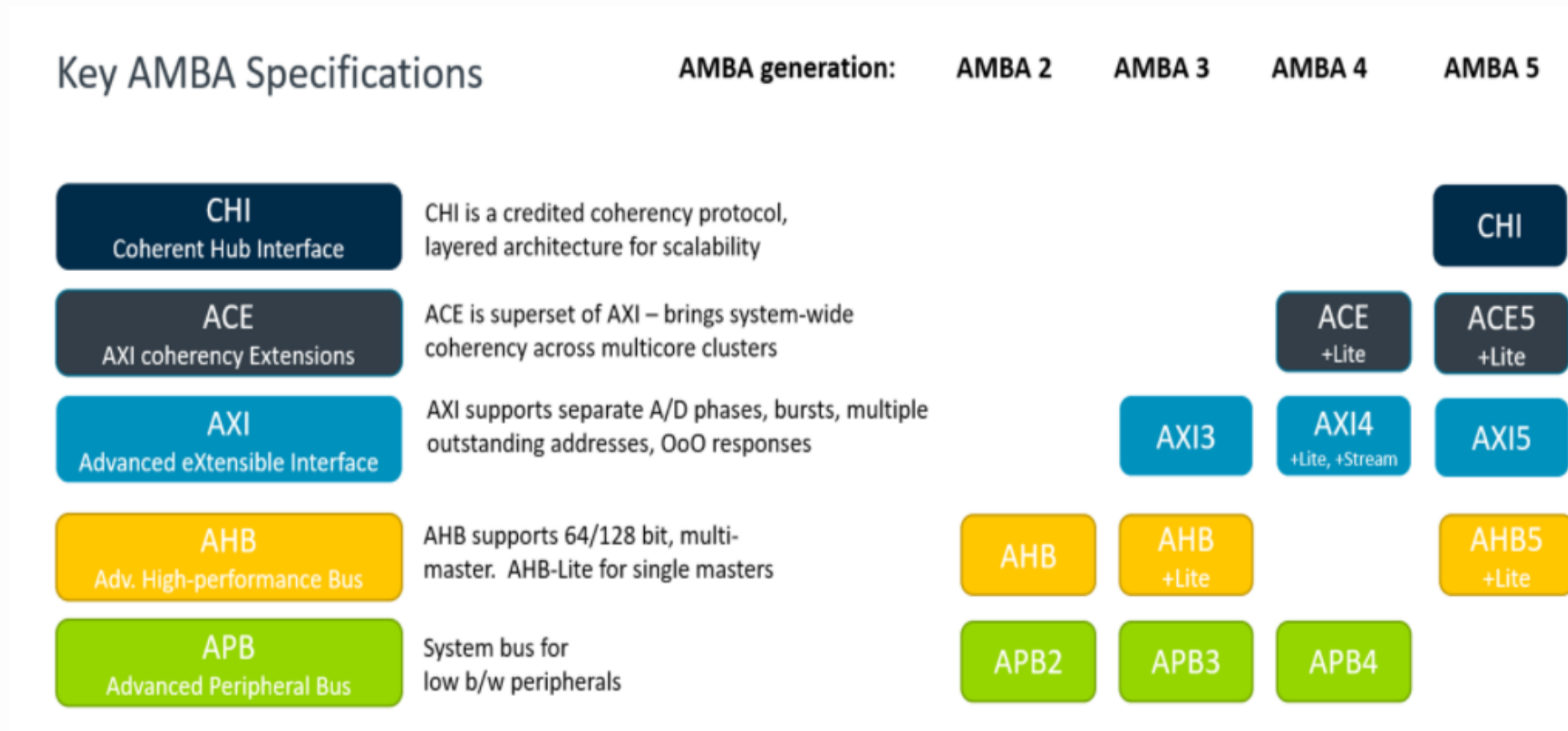
ADD Ra, Ra, Rj, LSL #2

This results in the typical ARM program being denser than expected with fewer memory accesses; thus the pipeline is used more efficiently.

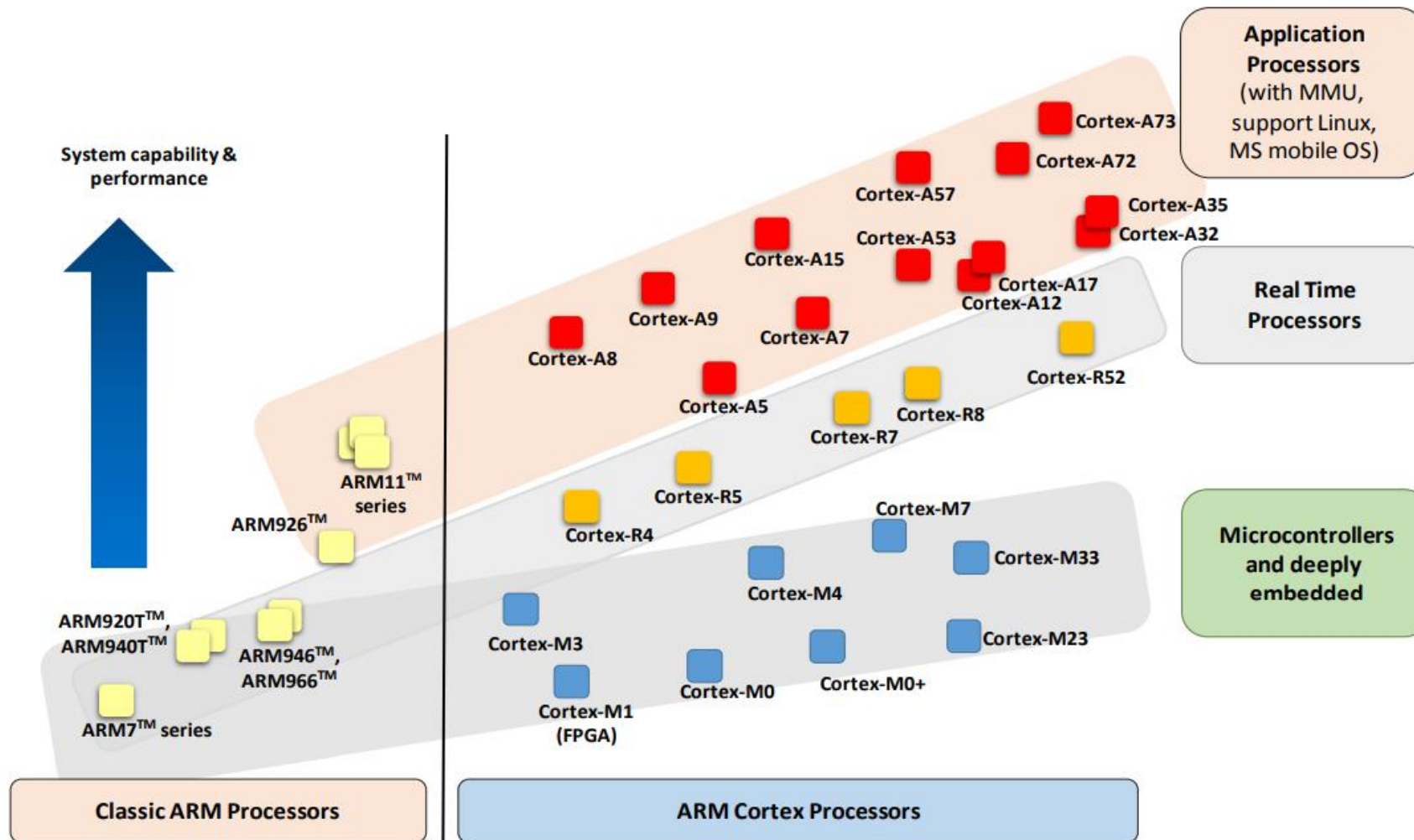
- Enhanced DSP/SIMD/VFP instructions are added to standard ARM instruction set to support faster operation.

Design Features contd..

- Advanced Microcontroller Bus Architecture (AMBA) has been widely used for ARM



ARM Processor Families



ARM Cortex Processor Families

- **A Profile** : Application processors which are designed to handle complex applications such as high-end embedded operating systems (OSs)
 - These processors requiring the highest processing power, virtual memory system support with memory management units (MMUs).
E.g: High-end mobile phones(Samsung S6: Samsung Exynos).
- **R Profile**: Real-time, high-performance processors targeted primarily at the higher end of the real-time market.
 - Those applications, such as high-end braking systems and hard drive controllers, in which high processing power and high reliability are essential and for which low latency is important.
- **M Profile** : Processors targeting low-cost applications in which processing efficiency is important and cost, power consumption, low interrupt latency, and ease of use are critical.

Typical Application Domains

Cortex - A

Highest performance

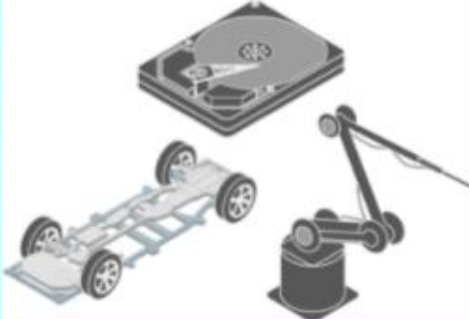
Optimised for rich operating systems



Cortex - R

Fast response

Optimised for high performance, hard real-time applications



Cortex - M

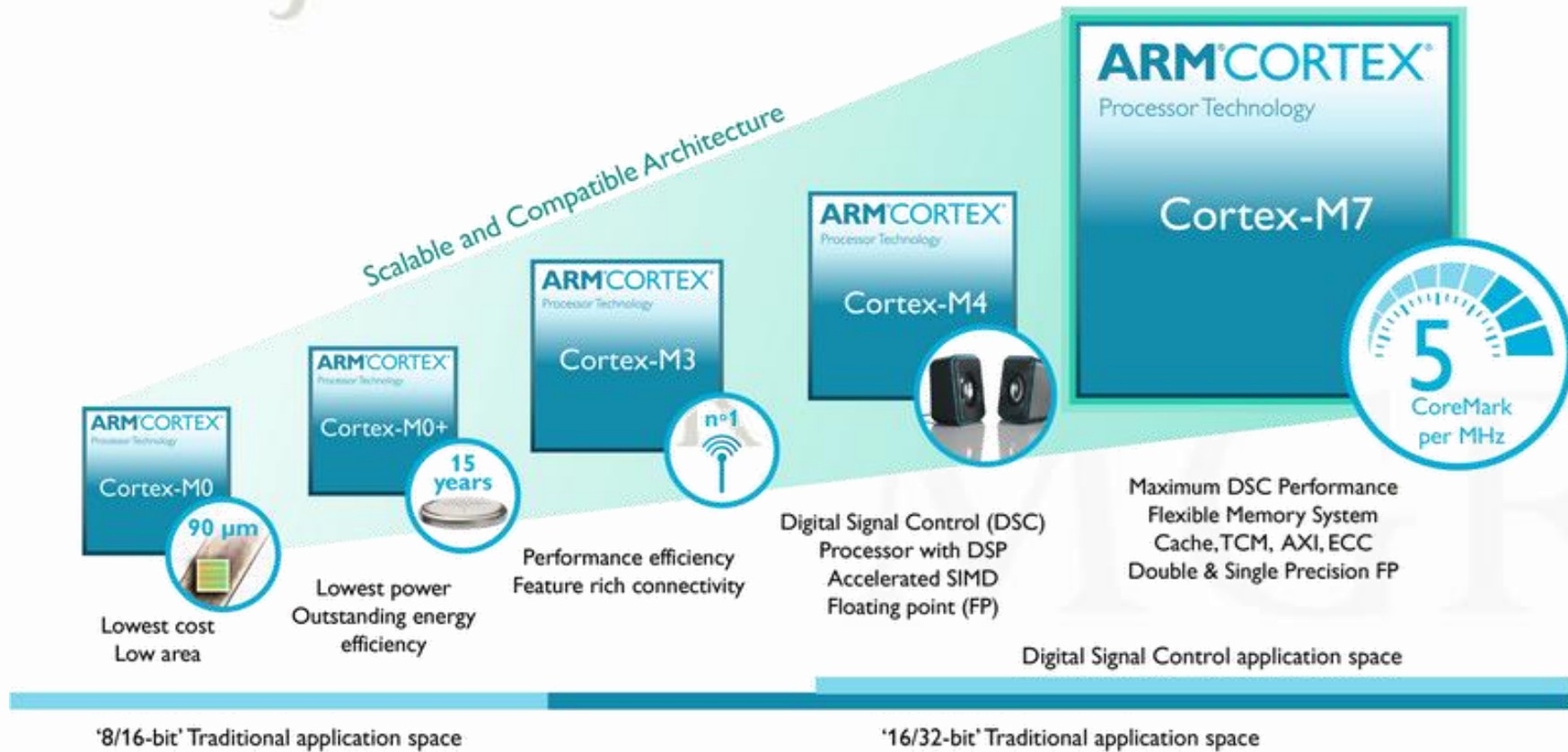
Smallest/lowest power

Optimised for discrete processing and microcontrollers



Source: google

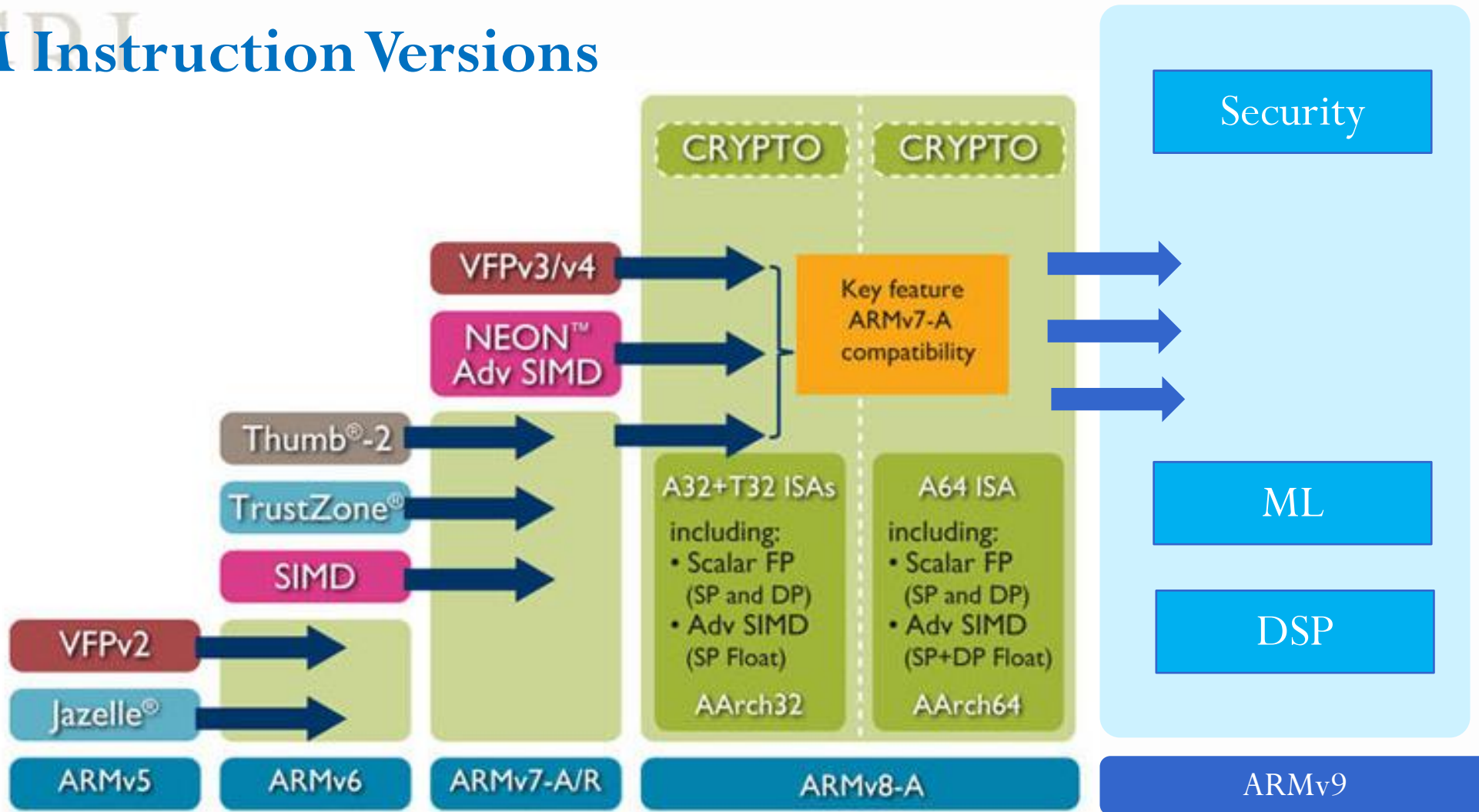
ARM Cortex-M Series



CoreMark

- CoreMark is small, portable, easy to understand, free, and displays a single number benchmark score to represent the speed of processors.
- A processor with higher CoreMark number is faster. This rating is given by CoreMark consortium.
- Earlier DMIPS(Dhrystone Million Instructions Per second) is the metrics used specify the speed of processors.
- To avoid the problems associated with the DMIPS, CoreMark rating is introduced.
- More info on CoreMark: <https://www.eembc.org/coremark/>

ARM Instruction Versions



ARMv7-M

ARMv8-R/M

www.arm.com

MGR,ECE,RVCE

Traditional Arm ISAs

- **ARM instruction set:** 32 bit in size
- **Thumb instruction set:** 16 bit

Code Density Comparison

ARM code

ARMDivide

; IN: r0(value),r1(divisor)
; OUT: r2(MODulus),r3(DIVide)

```
      MOV    r3,#0
loop  SUBS   r0,r0,r1
      ADDGE  r3,r3,#1
      BGE    loop
      ADD    r2,r0,r1
```

$5 \times 4 = 20$ bytes

Thumb code

ThumbDivide

; IN: r0(value),r1(divisor)
; OUT: r2(MODulus),r3(DIVide)

```
      MOV    r3,#0
loop  ADD    r3,#1
      SUB    r0,r1
      BGE    loop
      SUB    r3,#1
      ADD    r2,r0,r1
```

$6 \times 2 = 12$ bytes

- **Problem with ARM and Thumb: Interworking**
- BX instruction is to be executed transfer control from ARM mode the Thumb mode and to return to ARM mode.

```
; ARM code
CODE32                                ; word aligned
LDR    r0, =thumbCode+1              ; +1 to enter Thumb state
MOV     lr, pc                        ; set the return address
BX      r0                            ; branch to Thumb code & mode
; continue here

; Thumb code
CODE16                                ; halfword aligned
thumbCode
ADD     r1, #1
BX      lr                            ; return to ARM code & state
```

Traditional Arm ISAs

- Solution is Thumb-2: Mixture of 16 bit and 32 bit instructions.
- Microcontroller used in lab is based on Thumb-2.

Exp1 code segment

```
MOV    R0, #0x10
LDR    R1, =ROMARRAY2 + 28
LDR    R2, =ARRAY2 + 28

MEMCPY_LOOP
LDR    R3, [R1], #-4
STR    R3, [R2], #-4
SUBS   R0, R0, #1
BNE    MEMCPY_LOOP

MOV    R0, #LEN
LDR    R1, =ARRAY1+28
LDR    R2, =ARRAY2+28
```

Disassembly view

```
39:      MOV     R0, #0x10          ; HEX(16) =
0x08000010 F04F0010 MOV     r0,#0x10
40:      LDR     R1, =ROMARRAY2 + 28 ; Pc
0x08000014 490A      LDR     r1,[pc,#40] ; @0x0
41:      LDR     R2, =ARRAY2 + 28    ; Pc
42:
43: MEMCPY_LOOP
0x08000016 4A0B      LDR     r2,[pc,#44] ; @0x0
44:      LDR     R3, [R1], #-4
0x08000018 F8513904 LDR     r3,[r1],#-0x04
45:      STR     R3, [R2], #-4      ; I
0x0800001C F8423904 STR     r3,[r2],#-0x04
46:      SUBS    R0, R0, #1          ; Decrement th
0x08000020 1E40      SUBS    r0,r0,#1
47:      BNE     MEMCPY_LOOP        ; Loop t
48:
0x08000022 D1F9      BNE     0x08000018
49:      MOV     R0, #LEN
0x08000024 F04F0008 MOV     r0,#0x08
50:      LDR     R1, =ARRAY1+28
```